

Components of IoT Application

Gleb Bulygin

gbulygin@students.oamk.fi

DIN24SP

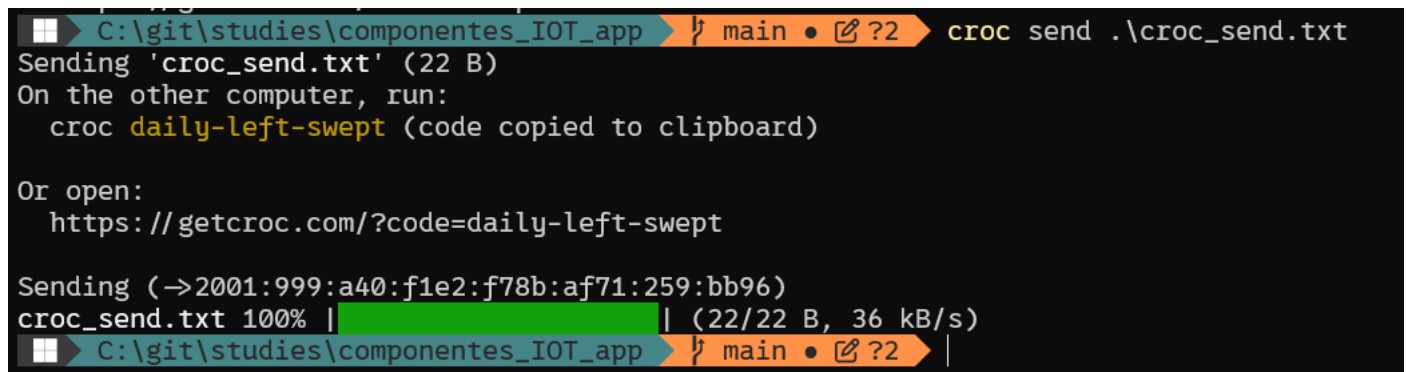
Autumn 2026

Week 4

26. Use **Croc** to move file or files between two or more hosts/devices. Answer shortly:

I have installed **croc** on Windows with `choco install croc` command (PowerShell as Admin).

I have installed **croc** on Android with `pkg install croc` (I use **termux** app).

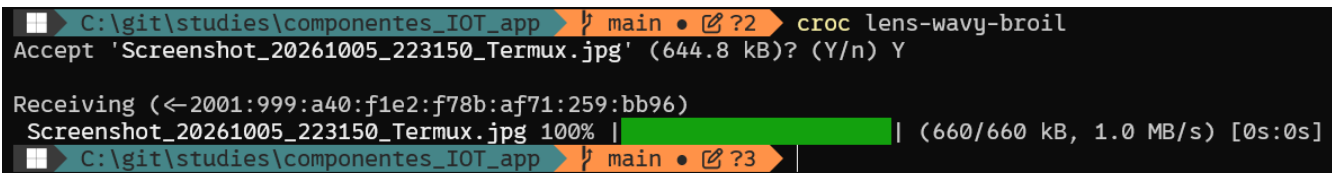


```
C:\git\studies\componentes_IOT_app > main • ?2 > croc send .\croc_send.txt
Sending 'croc_send.txt' (22 B)
On the other computer, run:
  croc daily-left-swept (code copied to clipboard)

Or open:
  https://getcroc.com/?code=daily-left-swept

Sending (->2001:999:a40:f1e2:f78b:af71:259:bb96)
croc_send.txt 100% | ██████████ | (22/22 B, 36 kB/s)
C:\git\studies\componentes_IOT_app > main • ?2 > |
```

Figure 4.1 — Sending files via **croc** from powershell



```

C:\git\studies\componentes_IOT_app > main • ?2 > croc lens-wavy-broil
Accept 'Screenshot_20261005_223150_Termux.jpg' (644.8 kB)? (Y/n) Y
Receiving (<2001:999:a40:f1e2:f78b:af71:259:bb96)
Screenshot_20261005_223150_Termux.jpg 100% | ██████████ | (660/660 kB, 1.0 MB/s) [0s:0s]
C:\git\studies\componentes_IOT_app > main • ?3 >

```

Figure 4.4 — Screenshot successfully received on Windows.

- **How the Croc works?**

Both sides pick/generate a one-time code phrase. This phrase is used in a **PAKE** (Password Authenticated Key Exchange) handshake, which lets sender and receiver derive the same secret encryption key without ever sending the phrase itself over the network. Croc then encrypts the file (AES-256/ChaCha20) and transfers it directly between the two hosts if they can reach each other, falling back to a relay server otherwise.

- **How the Croc moves files if both hosts are not directly visible to each other? (for example, both are behind NATs or basic firewalls)**

Both hosts only make **outbound** connections to a public (or self-hosted) **relay server** — no incoming ports or port-forwarding are needed. The relay server acts as a rendezvous point: it helps the two peers find each other and, if a direct peer-to-peer connection can't be made (common when both are behind NAT/firewalls), it simply pipes the already end-to-end encrypted traffic between them. Since the data is encrypted before it reaches the relay, the relay operator can't read the file contents.

27. Study how NTP protocol operates and analyse this [Python NTP client code](#). Also available here as [plain text](#).

- This Python script uses direct socket programming to access the NTP server. Comment individual socket programming related code lines. Also, answer these:

- What is the NTP server (DNS) hostname?

`pool.ntp.org` — the default `host` parameter of the `ntp_time()` function.

- What is the destination port number being used?

Port 123, the default `port` parameter of `ntp_time()` and the standard well-known port for the NTP protocol.

- Is this Python script using TCP or UDP? How do you know?

It's using **UDP**. The socket is created with `SOCK_DGRAM` (`socket(AF_INET, SOCK_DGRAM)`), which is the datagram/UDP socket type, as opposed to `SOCK_STREAM` which would indicate TCP. The code also uses `sendto()` / `recvfrom()` instead of `connect()` / `send()` / `recv()`, which is typical of connectionless UDP usage — the socket is never explicitly connected to the server before sending data.

- Try to execute the app with Python

```

C:\git\studies\componentes_IOT_app > python .\week4\src\Python_NTP_client_code.py
Mon Sep 28 21:18:25 2026
C:\git\studies\componentes_IOT_app >

```

Figure 4.5 — Python_NTP_client_code execution in the terminal

28. Do these Python programming assignments with Windows or Linux (or with MacOS if you want and know how)

- For example, use <https://realpython.com/python-sockets/> or similar site(s) for socket programming example codes and create TCP client and TCP server Python scripts
- Establish a TCP connection between your client and server Python scripts (either as localhost traffic or between two separate hosts if you have access to two or more Python running hosts without firewall preventing the traffic)
- Transfer some ASCII text strings between the hosts
 - TCP client connects to the server, sends some plain text string and then disconnects
 - Server prints the text to the console or elsewhere
 - Save your source codes and work. You need scripts again during the course week #5 (Wireshark protocol analyzer assignments)
- Use netstat or similar command line tools to check the TCP connection status (for example the Python server script LISTENING the selected TCP port)

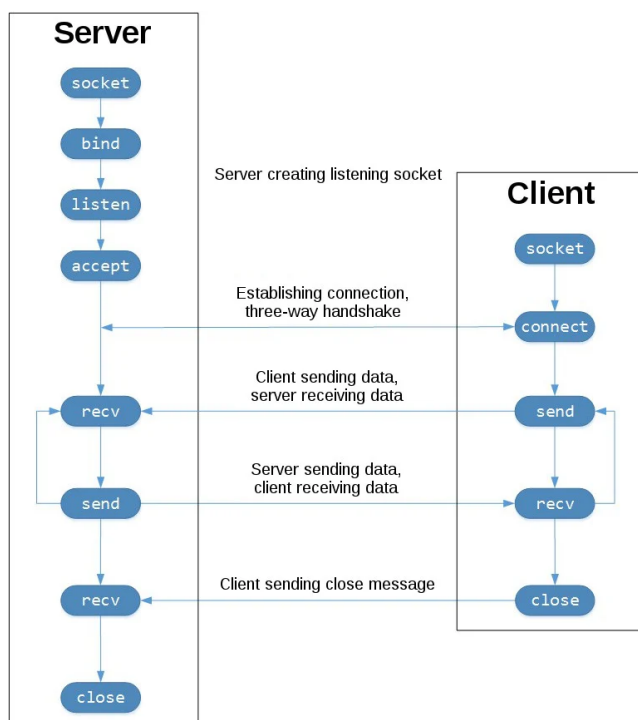


Figure 4.6 — sequence of socket API calls and data flow for TCP (source: <https://realpython.com/python-sockets/>)

I don't have access to another machine now, so my app will run on localhost. I have created two files `tcp_server.py` and `tcp_client.py`.

```
# tcp_server.py
import socket

HOST = "0.0.0.0" # Listen on all interfaces so a remote client can connect too
PORT = 65432

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
    server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server_socket.bind((HOST, PORT))
    server_socket.listen()
    print(f"Server listening on {HOST}:{PORT}")

    while True:
        conn, addr = server_socket.accept()
        with conn:
            print(f"Connected by {addr}")
            while True:
                data = conn.recv(1024)
                if not data:
                    break
                print(f"Received from {addr}: {data.decode('ascii')}")
            print(f"Disconnected: {addr}")

# tcp_client.py
import socket

HOST = "127.0.0.1" # localhost
PORT = 65432
MESSAGE = "Hello from the TCP client!"

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
    client_socket.connect((HOST, PORT))
    client_socket.sendall(MESSAGE.encode("ascii"))
    print(f"Sent: {MESSAGE}")
```

Figure 4.7 — a line of text sent from the client and received by the server

Protocol	Local Address	Foreign Address	State	PID
TCP	0.0.0.0:65432	0.0.0.0:0	LISTENING	49288

Figure 4.8 — tcp_server.py app is listening for all incoming connections on port 65432

After I stop the server app the command above returns nothing.